

5. Functional Processing of Collections (Advanced)

Author -- Yucheng Wu

Version -- 26.1.25

Example 5.1

```
import java.util.*;
import java.util.stream.*;

public class Example {
    public static void main(String[] args) {

        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);

        numbers.stream()                // 1. create a stream
            .filter(x -> x % 2 == 0)    // 2. keep even numbers
            .map(x -> x * 10)           // 3. transform each number
            .forEach(x -> System.out.println(x));
        // 4. consume each result

        int sum =
            numbers.stream()
                .filter(x -> x > 3)
                .reduce(0, (a, b) -> a + b);

        System.out.println(sum); // 18
    }
}
```

Example 5.2

```
List<String> words = Arrays.asList("apple", "banana", "pear");

long count =
    words.stream()
        .filter(s -> s.length() > 4)
        .count();
```

5.1. Functional Programming

Definition

Functional programming is a way of programming where we use functions to describe computation instead of changing variables step by step.

Features

- Focus on **what result we want**, not how to loop

- Avoids changing variables when possible
- Uses functions as values

Example

- mapping x to x+1.

```
numbers.stream().map(x -> x + 1);
```

- This describes a transformation, not a loop.

Remark

- Java is **not a pure functional language**
 - Java supports functional programming mainly through **lambda and streams**
 - Exam focus: understanding the style difference from `for` loops
-

5.2. Lambda Expression

Definition

The lambda expression is a simple representation of a method, bearing a strong similarity with simple methods or simple functions.

(Not important introduce later) Every lambda expression is essentially: an implementation of a unique abstract method in a certain functional interface.

Feature

They have

- A return type – Parameters – A body
- They don't have a name (anonymous methods)
- They have no associated object
- They can be passed as parameters: – As code to be executed by the receiving method

Syntax

```
(parameters) -> result
```

- A lambda expression tells Java **what to do with one input element**.

Example

```
x -> x * x
```

- Input: x
- Output: x * x

```
x -> System.out.println(x)
```

- print each x.

A Method and a lambda equivalent:

```
public void printSighting(Sighting record {  
    System.out.println(record.getDetails());  
}
```

```
(Sighting record) ->  
{  
    System.out.println(record.getDetails());  
}
```

Remark

- Lambdas **do not work alone**
 - They must be used with a **functional interface** (I will introduce it later)
 - At this moment, the lambda expression are used in Stream.
-

5.3. Stream

Definition

A stream is a sequence of elements that can be processed one by one.

Example

- A stream is usually created from a collection using `collection.stream()`.

```
numbers.stream()
```

```
words.stream()
```

- No computation happens yet.
- A stream does not store data and does not change the original collection.

Features

- Reads data from a source
- Processes data step by step
- Can be used only once

Remark

- A stream is **not** a list
- It is a **process**, not a container

5.4. Pipeline

Definition

A pipeline is a chain of stream operations written in one expression.

Features

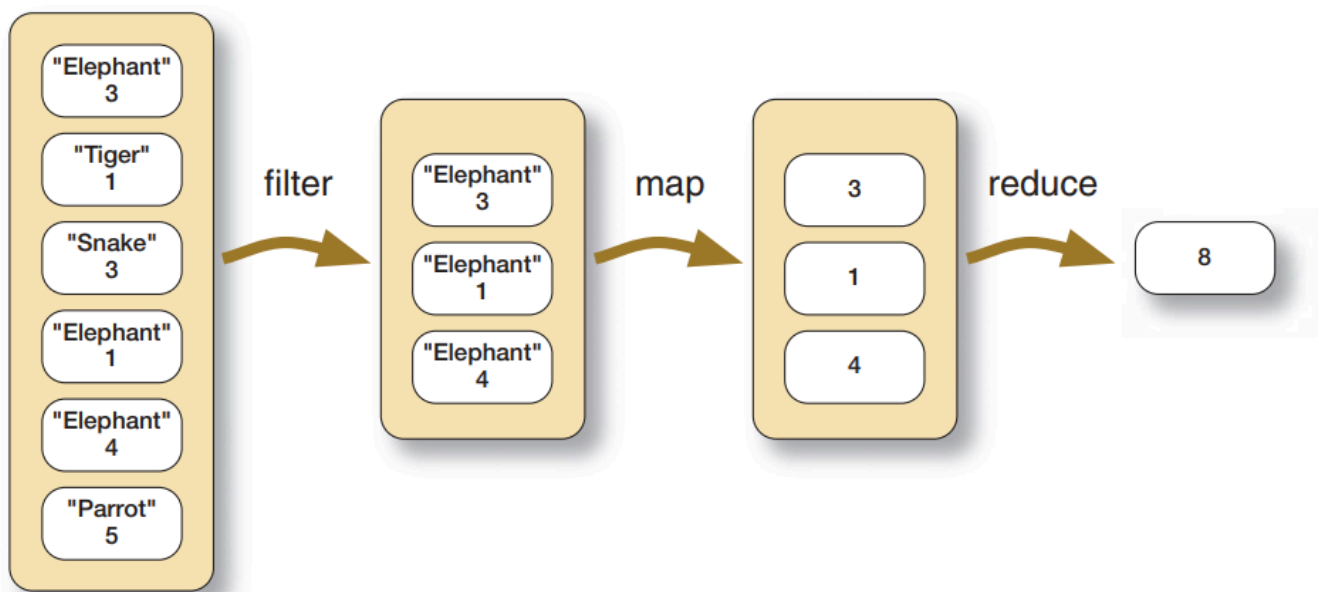
A pipeline contains:

1. Intermediate Operations

- Operations that return a stream and do not produce a final result (Intermediate Operations are lazy).

2. Terminal Operation

- The last operation that produces a result and starts execution.



Example

```
numbers.stream()
    .filter(x -> x > 3)
    .map(x -> x * 2)
    .count();
```

- Execution starts only at the terminal operation: `count()` starts the pipeline.
- Operations are connected using dots

Remark

- Pipelines are common in exams for step-by-step tracing

5.5. Tool in Stream

General Example

- These two are equivalent

```
public int getCount(String animal) {  
    return sightings.stream()  
        .filter(sighting -> animal.equals(sighting.getAnimal()))  
        .map(sighting -> sighting.getCount())  
        .reduce(0, (total, count) -> return total + count);  
}
```

```
public int getCount(String animal) {  
    int total = 0;  
    for(Sighting sighting : sightings) {  
        if(animal.equals(sighting.getAnimal())) {  
            total = total + sighting.getCount();  
        }  
    }  
    return total;  
}
```

Remark

Operation	Type
filter	Intermediate operation
map	Intermediate operation
forEach	Terminal operation
reduce	Terminal operation

5.5.1. filter

Definition

`filter` keeps elements that satisfy a condition.

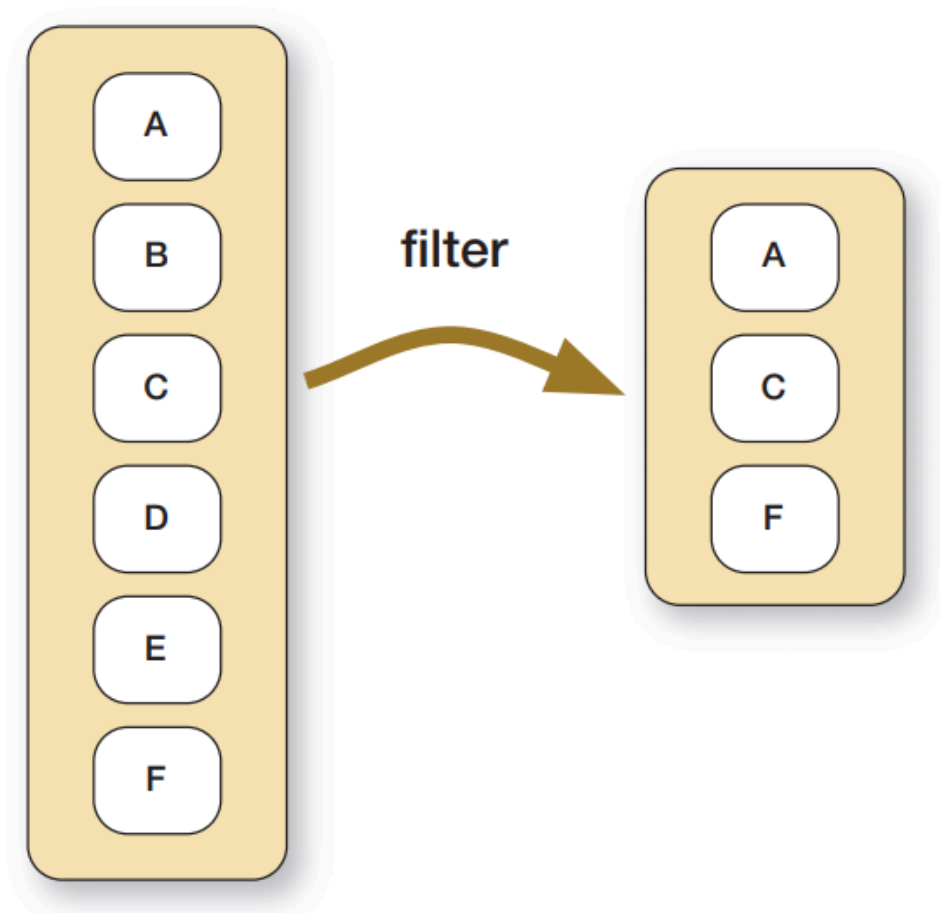
Syntax

```
.filter(x -> boolean expression)
```

Features

- Does not change elements

- Can reduce the number of elements



Example

```
.filter(x -> x % 2 == 0)
```

- Keeps only even numbers.

Remark

- If the condition is false, the element is removed

5.5.2. map

Definition

`map` changes each element into a new value.

Syntax

```
.map(x -> f(x))
```

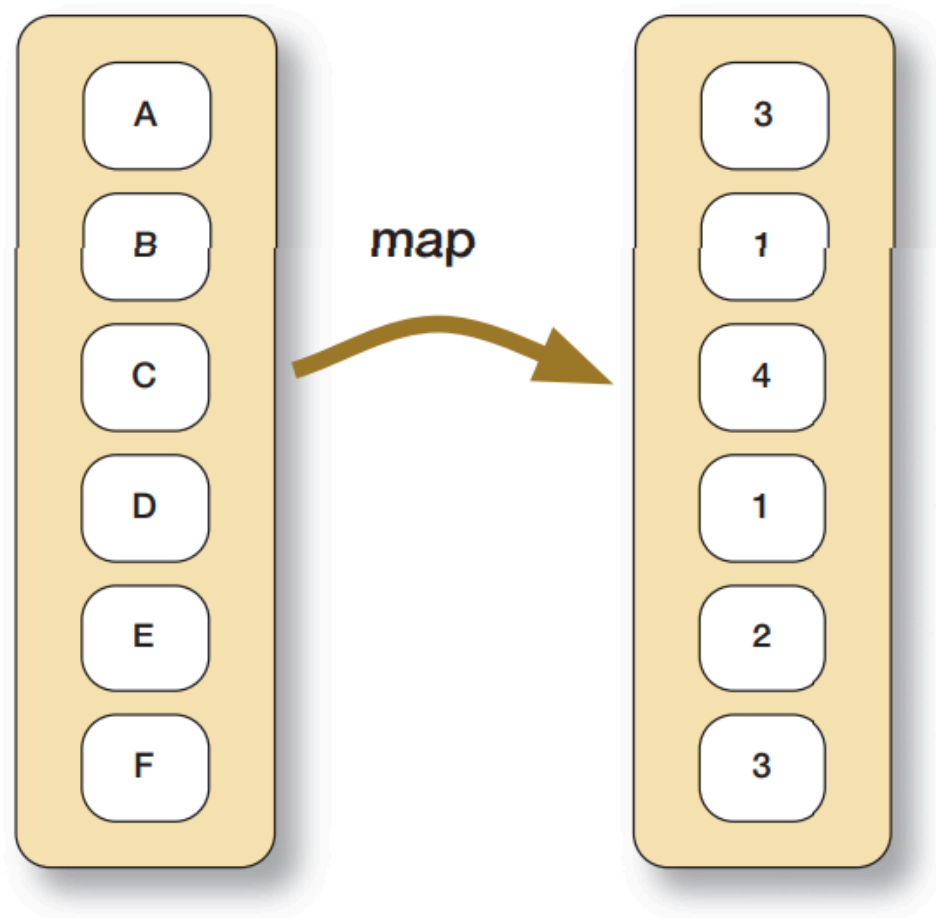
Features

1. One-to-One Rule

- Each input element produces exactly one output element.

2. Behave as a function

- Changes values
- Does not change the number of elements



Example

```
.map(x -> x * x)
```

- Each number becomes its square.

Remark

- `map` transforms data, it does not remove data

5.5.3. forEach

Definition

`forEach` performs an action on each element and ends the stream.

Features

```
.forEach(x -> System.out.println(x));
```

- Terminal operation
- Returns no result

Example

Processing a collection – the usual approach

```
loop (for each element in the collection):  
    get one element;  
    do something with the element;  
end loop
```



```
for (Sighting record : sightings) {  
    printSighting(record);  
}
```

Processing a whole collection

```
collection.doThisForEachElement(some code);
```



```
sightings.forEach((Sighting record) ->  
    {  
        System.out.println(record.getDetails());  
    }  
);
```

Reduced lambda syntax

Type Inference

```
sightings.forEach((record) ->  
    {  
        System.out.println(record.getDetails());  
    }  
);
```

Single parameter

```
sightings.forEach(record ->
    {
        System.out.println(record.getDetails());
    }
);
```

Single statement

```
sightings.forEach(
    record -> System.out.println(record.getDetails())
);
```

for-each loop vs. forEach()

```
for(String name : students) {
    System.out.println(name);
}
```

```
students.forEach(name -> System.out.println(name));
```

Remark

- Used for printing, logging, side effects
- Cannot be followed by other operations

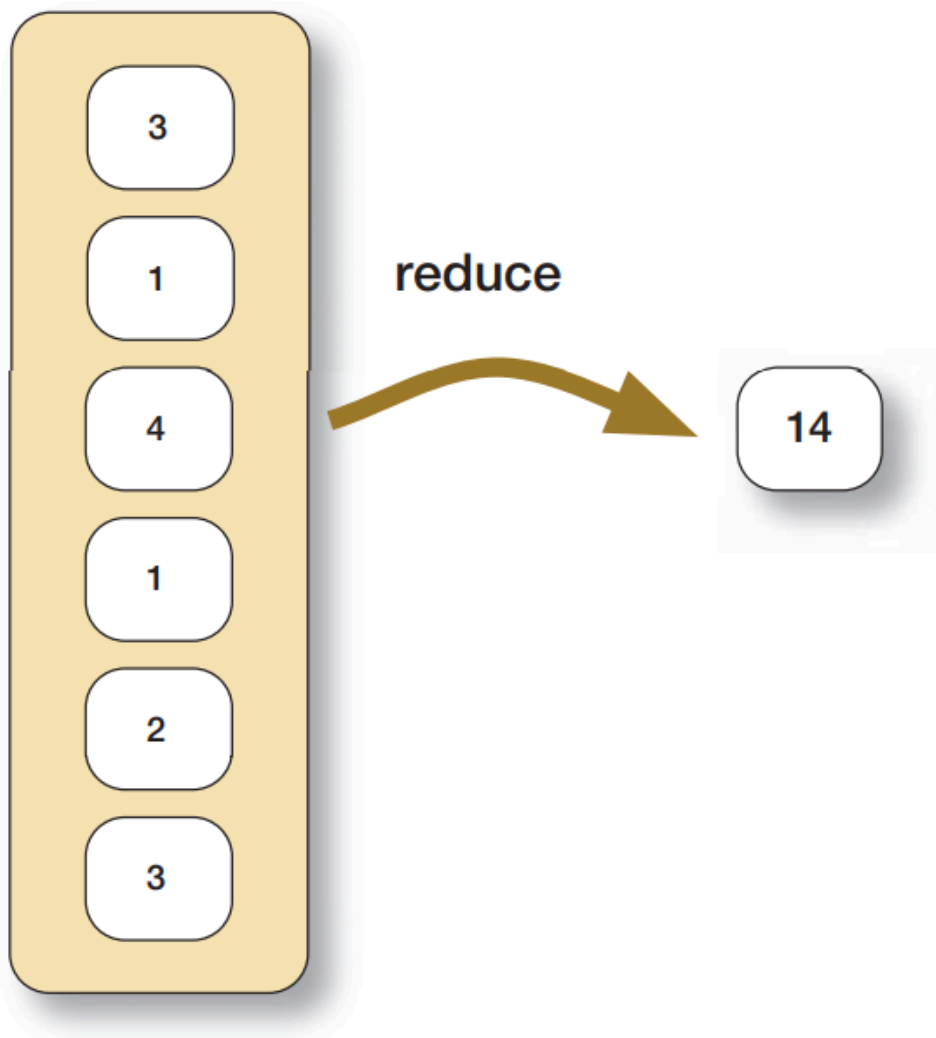
5.5.4. reduce

Definition

`reduce` combines all elements into one single result.

Features

- Produces one result
- Often used for sum, product, maximum
- Terminal operation



`reduce` contains:

1. Identity

- The identity is the initial value.

2. Accumulator

- The accumulator tells Java how to combine two values.

Example

```
.reduce(0, (a, b) -> a + b)
```

- **Identity:** 0 (start from 0)
- **Accumulator:** (a, b) -> a + b (add each element)

Remark

- The identity must not change the result
(e.g. 0 for sum, 1 for product)